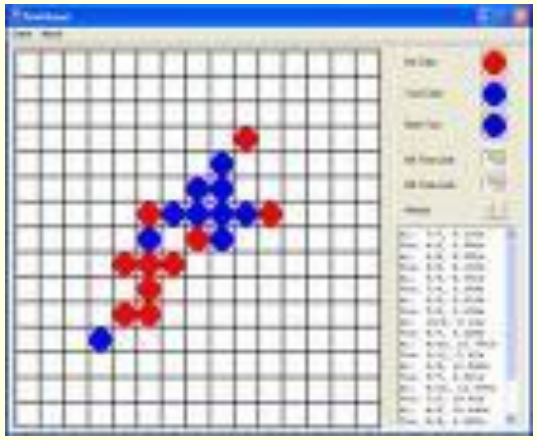





# Artificial Intelligence
# (Part 2b)
# Knowledge Representation and Search:
# **PREDICATE LOGIC**

# Course Contents

Again..Selected topics for our course. Covering all of AI is impossible!

Key topics include:

- Introduction to Artificial Intelligence (AI)
- Knowledge Representation and Search
- Introduction to AI Programming
- Problem Solving Using Search
- Exhaustive Search Algorithm
- Heuristic Search
- Techniques and Mechanisms of Search Algorithm
- Knowledge Representation Issues and Concepts
- Strong Method Problem Solving
- Reasoning in Uncertain Situations
- Soft Computing and Machine Learning

# Basic concepts of logic

- syntax: formal structure of sentences
- semantics: truth of sentences wrt models
- entailment: necessary truth of one sentence given another
- inference: deriving sentences from other sentences
- soundness: derivations produce only entailed sentences
- completeness: derivations can produce all entailed sentences

# Recall: Propositional Logic

- Logic is a form of knowledge representation
- Sentences have to obey syntactic laws
- Truth is established with respect to a *model* of the world and an *interpretation* which maps symbols to world objects
- Propositional logic is a very simple form of logic
- The truth table method is a sound and complete inference method, which checks truth in all possible models.
- But the truth table method is very inefficient! $2^n$ models for $n$ literals
- Today: inference methods that use *syntactic operations* on sentences

# First-Order Logic (FOL)

- FOL is a formal language for describing knowledge about objects, their properties and relations
- Constants and variables are associated with objects, while predicates describe relations between objects
- Unlike propositional logic, it can handle infinite domains through the use of quantifiers
- Generalized modus ponens is a sound inference method, but it is complete only for a subset of FOL (clauses with exactly one positive literal)
- Inference in FOL is only semi-decidable: can find a proof if one exists, but if a proof does not exist, the procedure may not halt.

# First-Order Logic (FOL)

- A key element of FOL are *predicates*, which are used to describe objects, properties, and relationships between objects
  E.g. *On(x,y)*
- A *quantified statement* is a statement that applies to a class of objects
  E.g. $\forall x\ On(x, Table) \rightarrow Fruit(x)$
  - This means that there is only fruit on the table
  - The first element is called a *quantifier*, *x* is a *variable* and *Table* is a *constant*
  - *On* is a *predicate*
- The use of *quantifiers* allows FOL to handle *infinite domains*, while propositional logic can only handle finite domains.

# First Order Predicate Logic

- Includes 2 symbols:
  - Variable quantifiers
    - ∃ (existential) and
    - ∀ (universal)
- A quantifier followed by a variable and a sentence:
  - ∀ X likes(X,pizza) ; true for all X
  - ∃ Y friends(Y,amir) ; true if there is atleast one

# Universal Quantification

- Syntax: $\forall$ *variables sentence*
- E.g. Everyone taking AI is smart.

  $\forall x$ *Taking(x,AI)* $\rightarrow$ *Smart(x)*
- Semantics: $\forall x$ *S* is equivalent to the *conjunction of instantiations* of *S*:

  *Taking(John,AI)* $\rightarrow$ *Smart(John)*

  $\wedge$ *Taking(Ann,AI)* $\rightarrow$ *Smart(Ann)*

  $\wedge$ ...
- Typically, $\rightarrow$ is the main connective with $\forall$.

# Properties of Quantifiers

- $\forall x \forall y$ is the same as $\forall y \forall x$
- $\exists x \exists y$ is the same as $\exists y \exists x$
- $\exists x \forall y$ *is not the same as* $\forall y \exists x$ ????

$\exists x \forall y Loves(x, y)$

$\forall y \exists x Loves(x, y)$

# Properties of Quantifiers

- $\forall x \forall y$ is the same as $\forall y \forall x$
- $\exists x \exists y$ is the same as $\exists y \exists x$
- $\exists x \forall y$ *is not the same as* $\forall y \exists x$

  $\exists x \forall y Loves(x, y)$

  "There is a person who loves everyone in the world"

  $\forall y \exists x Loves(x, y)$

  "Everyone in the world is loved by at least one person"

# Quantifier Duality

Each quantifier can be expressed using the other quantifier and negation:

- $\forall x\ Likes(x, IceCream)$ is equivalent to $\neg\exists x\ \neg Likes(x, IceCream)$
- $\exists x\ Likes(x, Broccoli)$ is equivalent to $\neg\forall x\ \neg Likes(x, Broccoli)$

# Fun with Sentences

- Brothers are siblings

$$\forall x \forall y Brother(x, y) \rightarrow Sibling(x, y)$$

- "Sibling" is reflexive

$$\forall x \forall y Sibling(x, y) \leftrightarrow Sibling(y, x)$$

- One's mother is one's female parent

$$\forall x \forall y Mother(x, y) \leftrightarrow (Female(x) \wedge Parent(x, y))$$

- A first cousin is a child of a parent's sibling

$$\forall x \forall y FirstCousin(x, y) \leftrightarrow \exists p \exists ps Parent(p, x) \wedge Sibling(ps, p) \wedge Parent(ps, y)$$

# 2.2 Predicate Calculus (13)

## Definition - First-order Predicate Calculus

- ***First-order predicate calculus*** **allows quantified variables to refer to objects in the domain of discourse and not to predicates or functions.**
- Examples of representing English sentence
  - **If it doesn't rain tomorrow, Tom will go to the mountains**
    - **$\neg$weather(rain, tomorrow) $\Rightarrow$ go(tom, mountains)**
  - **Emma is a Doberman pinscher and a good dog**
    - **gooddog(emma) $\wedge$ isa(emma, doberman)**
  - **All basketball players are tall**
    - **$\forall$ X (basketball_player(X) $\Rightarrow$ tall(X))**
  - **If wishes were horses, beggars would ride.**
    - **equal(wishes, horses) $\Rightarrow$ ride(beggars).**
  - **Nobody likes taxes**
    - **$\neg\exists$ X likes(X, taxes)**

# Try this…represent in Predicate Logic

- If it doesn't rain on Monday, Naim will go to the mountain
- All children are cute
- Nobody likes mouse

$\neg$ weather (rain, Monday) $\Rightarrow$ go(Naim,mountain)

$\forall$ X (children(X) $\Rightarrow$ cute(X))

$\neg\exists$ X likes(X,mouse)

# Proof methods

Proof methods divide into (roughly) two kinds:

- *Model checking*:
  - Truth table enumeration (sound and complete for propositional logic)
  - Heuristic search in model space (sound but incomplete)
- *Application of inference rules*:
  - Legitimate (sound) generation of new sentences from old
  - A *proof* is a sequence of inference rule applications
  - Inference rules can be used as operators in a standard search algorithm!

# Proofs

The proof process can be viewed as a *search* in which the operators are *inference rules*:

- *Modus Ponens (MP)*

$$\frac{\alpha, \quad \alpha \rightarrow \beta}{\beta} \qquad \frac{\textit{Takes(Joe,AI)} \quad \textit{Takes(Joe,AI)} \rightarrow \textit{Cool(Joe)}}{\textit{Cool(Joe)}}$$

- *And-Introduction (AI)*

$$\frac{\alpha \quad \beta}{\alpha \wedge \beta} \qquad \frac{\textit{Cool(Joe)} \quad \textit{CSMajor(Joe)}}{\textit{Cool(Joe)} \wedge \textit{CSMajor(Joe)}}$$

- *Universal Elimination (UE)*

$$\frac{\forall x \alpha}{\alpha\{x/\tau\}} \qquad \frac{\forall x \; \textit{Takes(x,AI)} \rightarrow \textit{Cool(x)}}{\textit{Takes(Pat,AI)} \rightarrow \textit{Cool(Pat)}}$$

$\tau$ must be a ground term: a term with no variables

# Example Proof

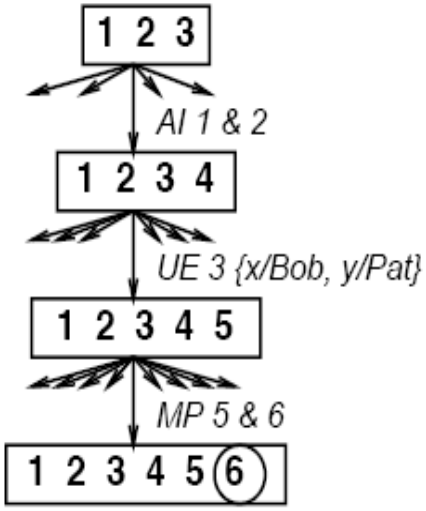
| | |
|---|---|
| Bob is a buffalo | 1. Buffalo(Bob) |
| Pat is a cat | 2. cat(Pat) |
| Buffaloes outrun cat | 3. $\forall x \forall y$ Buffalo(x) $\wedge$ cat(y) $\rightarrow$ Faster(x,y) |
| AI 1 & 2 | 4. Buffalo(Bob) $\wedge$ cat(Pat) |
| UE 3, x/Bob, y/Pat | 5. Buffalo(Bob) $\wedge$ cat(Pat) $\rightarrow$ Faster(Bob,Pat) |
| MP 4 & 5 | 6. Faster(Bob,Pat) |

# Search with Primitive Inference Rules

Operators are inference rules

States are sets of sentences

Goal test checks state to see if it contains query sentence



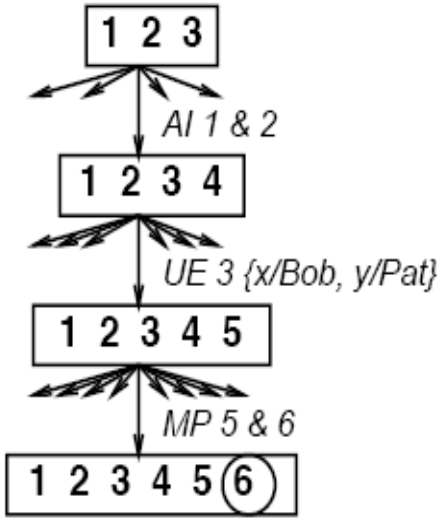

AI, UE, MP is a common inference pattern

# Search with Primitive Inference Rules

Operators are inference rules

States are sets of sentences

Goal test checks state to see if it contains query sentence




AI, UE, MP is a common inference pattern

Problem: branching factor huge, especially for UE

Idea: find a substitution that makes the rule premise match some known facts

$\Rightarrow$ a single, more powerful inference rule

# Unification

- A *substitution* $\sigma$ unifies atomic sentences $p$ and $q$ if $p\sigma = q\sigma$

| $p$ | $q$ | $\sigma$ |
|---|---|---|
| *Knows(John,x)* | *Knows(John,Jane)* | *x/Jane* |
| *Knows(John,x)* | *Knows(y,Mary)* | *y/John,x/Mary* |
| *Knows(John,x)* | *Knows(y,Mother(y))* | *y/John,x/Mother(John)* |

- Idea: Unify rule premises with known facts, apply unifier to conclusion
- E.g., if we know *q* and the rule: *Knows(John,x)* $\rightarrow$ *Likes(John,x)*, we conclude:
  - *Likes(John,Jane)*
  - *Likes(John,Mary)*
  - *Likes(John,Mother(John))*

# The unification algorithm

**function** UNIFY($x, y, \theta$) **returns** a substitution to make $x$ and $y$ identical
  **inputs**: $x$, a variable, constant, list, or compound
    $y$, a variable, constant, list, or compound
    $\theta$, the substitution built up so far

  **if** $\theta$ = failure **then return** failure
  **else if** $x = y$ **then return** $\theta$
  **else if** VARIABLE?($x$) **then return** UNIFY-VAR($x, y, \theta$)
  **else if** VARIABLE?($y$) **then return** UNIFY-VAR($y, x, \theta$)
  **else if** COMPOUND?($x$) **and** COMPOUND?($y$) **then**
    **return** UNIFY(ARGS[$x$], ARGS[$y$], UNIFY(OP[$x$], OP[$y$], $\theta$))
  **else if** LIST?($x$) **and** LIST?($y$) **then**
    **return** UNIFY(REST[$x$], REST[$y$], UNIFY(FIRST[$x$], FIRST[$y$], $\theta$))
  **else return** failure

# The unification algorithm

**function** UNIFY-VAR($var, x, \theta$) **returns** a substitution

**inputs**: $var$, a variable

$x$, any expression

$\theta$, the substitution built up so far

**if** $\{var/val\} \in \theta$ **then return** UNIFY($val, x, \theta$)
**else if** $\{x/val\} \in \theta$ **then return** UNIFY($var, val, \theta$)
**else if** OCCUR-CHECK?($var, x$) **then return** failure
**else return** add $\{var/x\}$ to $\theta$

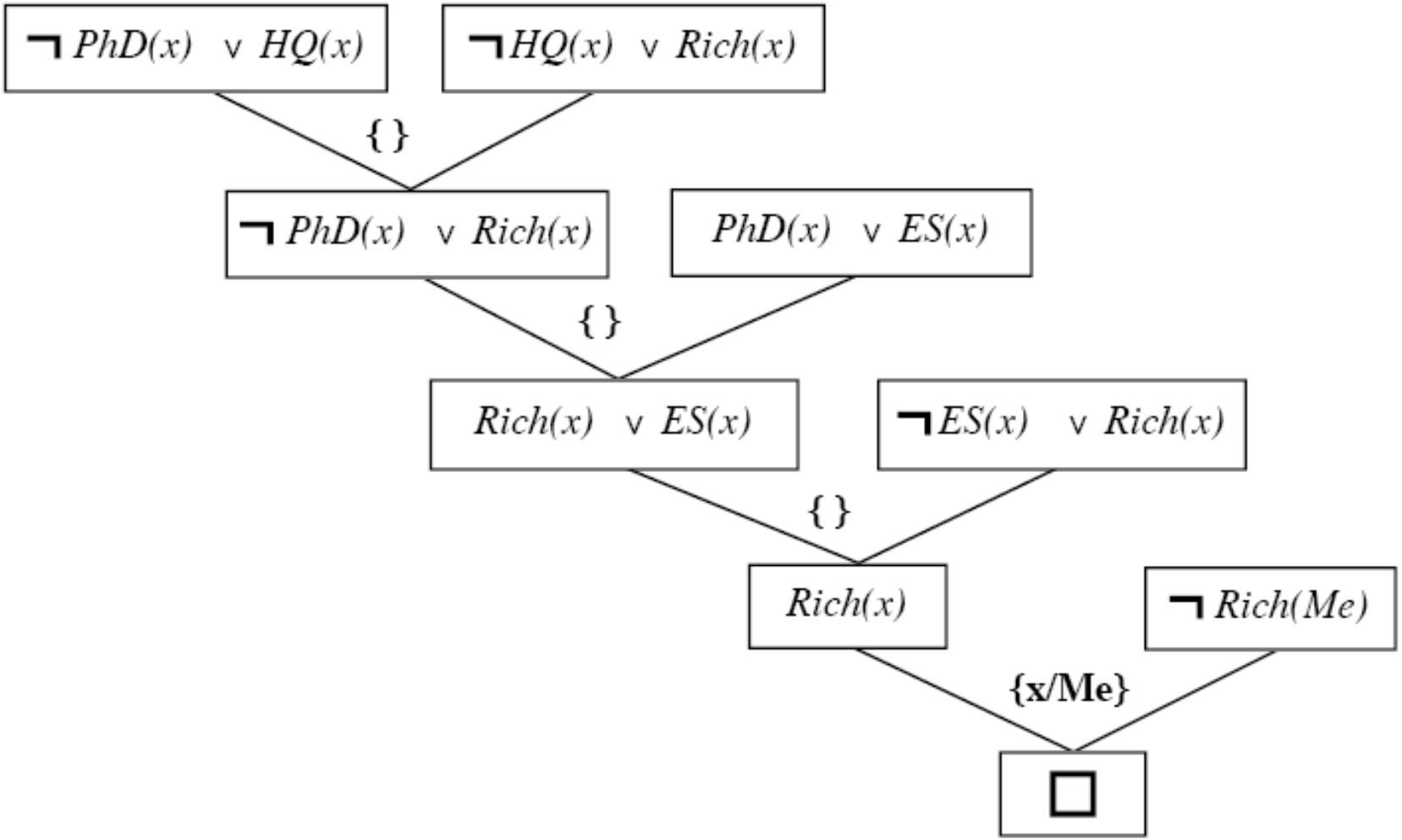
# Resolution

- Resolution is a sound and complete inference method for first-order logic.
- Resolution is a *refutation* procedure: to prove that $KB \models \alpha$, we show that $KB \wedge \neg\alpha$ is unsatisfiable
- The knowledge base and $\neg\alpha$ are expressed in universally quantified, conjunctive normal form
- Like in propositional logic, the resolution inference rule combines two clauses to make a new one:




- Inference continues until an empty clause is derived (contradiction)

# Resolution Proof Tree



# Resolution Strategies

Heuristics that impose a sensible order on the resolutions we attempt:

- *Unit resolution*: prefer to perform resolution if one clause is just a literal - yields shorter sentences
- *Set of support*: identify a subset of the *KB* (hopefully small); every resolution will take a clause from the set and resolve it with another sentence, then add the result to the set of support
  - *Can make inference incomplete!*
- *Input resolution*: always combine a sentence from the query or *KB* with another sentence
  - Modus ponens is a kind of input resolution
  - Not compete in general

# Example: Translate the KB into Propositional Logic

- If it is hot and humid, then it is raining. If it is humid, then it is hot. It is humid.

| | |
|---|---|
| H | It is hot. |
| D | It is humid. |
| R | It is raining. |

1. If it is hot and humid, then it is raining
2. If it is humid, then it is hot
3. It is humid

# Example: PROOF-Logical Inference Rules

GOAL-Is it Raining?

1. (H ^ D) → R
2. D → H
3. D

From 2 and 3: by Modus Ponens, we infer:

4. H

From 4: by ^-introduction, we infer:

5. H ^ D

From 1 and 5: by Modus Ponens, we infer:

6. **R** (Goal -- It is raining)

# Applications of First-Order Logic

- Prolog: a logic programming languages
- Production systems
- Semantic nets
- Automated theorem proving
- Planning

# Summary

- First-order logic:
  - objects and relations are semantic primitives
  - *syntax*: constants, functions, predicates, equality, quantifiers
- Increased expressive power

# Next..

- Programming in Prolog

# Translate into Predicate Logic:

- 1. If it doesn’t rain today, I will go to the class.
- 2. Putih is a siamese and a good cat.
- 3. All basketball players are tall.
- 4. Some people like reading.
- 5. I have a brother who is a teacher.